Project - High Level Design

on Chat with

Logistics Document (RAG)

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	REET SOOD	EN22CS301796
2.	PARTH AKHAND	EN22CS301687
3.	ROSHNI RAWAT	EN22CS301826
4.	PRAKRATI RASANIYA	EN22CS301722
5.	RIYANSHI JAIN	EN22CS301816
6.	RISHI YADAV	EN22CS301806

Group Name: Group 02D7

Project Number: GAI-25

Industry Mentor Name: Aashruti Shah Ma'am

University Mentor Name: Dr. Hemlata Patel Ma'am

Academic Year: 2022-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

The *Chat with Logistics Document (RAG)* system is an intelligent document understanding and question-answering platform designed specifically for logistics and supply chain documents. The system leverages Retrieval-Augmented Generation (RAG) to enable users to interact with large volumes of unstructured data such as reports, manuals, and shipment records.

Traditional document analysis requires manual searching, which is time-consuming and inefficient. This system automates the process by converting documents into machine-understandable formats and enabling semantic querying.

The system ensures that all generated responses are **context-aware, accurate, and grounded strictly in the uploaded documents**, thereby eliminating hallucination and improving reliability.

1.1 Scope of the Document

This document defines the high-level architecture and design of the system. It includes:

- Overall system architecture
- Component interaction and workflow
- Data handling and storage mechanisms
- Interface definitions
- Non-functional requirements

This document does not include:

- Detailed code implementation
- Deployment configurations
- Infrastructure-level design

1.2 Intended Audience

- Software Developers
- System Architects

- Project Evaluators
- Researchers working on NLP and RAG systems
- Students studying AI-based document systems

1.3 System Overview

The system allows users to upload logistics documents (PDFs) and query them using natural language. The system retrieves relevant document content and generates responses using a Large Language Model (LLM).

🔸 Core Concept: Retrieval-Augmented Generation (RAG)

The system follows a **Retrieval-Augmented Generation (RAG)** architecture, which separates knowledge retrieval from response generation.

- **Retrieval Phase:** Relevant document chunks are fetched using semantic similarity
- **Generation Phase:** LLM generates responses using retrieved context

This ensures:

- High accuracy
- No hallucination
- Domain-specific responses

2. System Design

The system follows a **layered architecture** ensuring modularity and scalability.

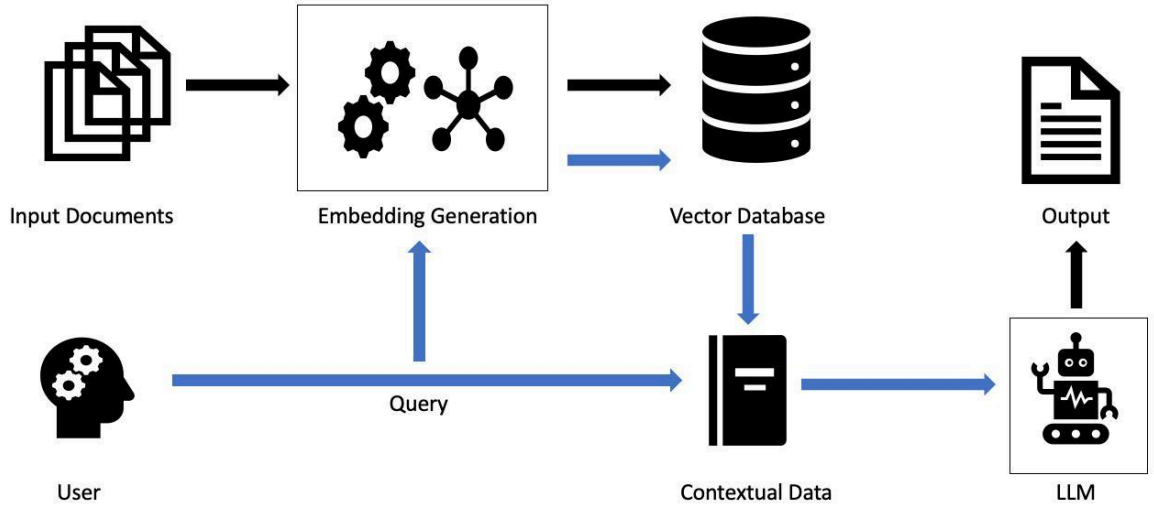
Layers:

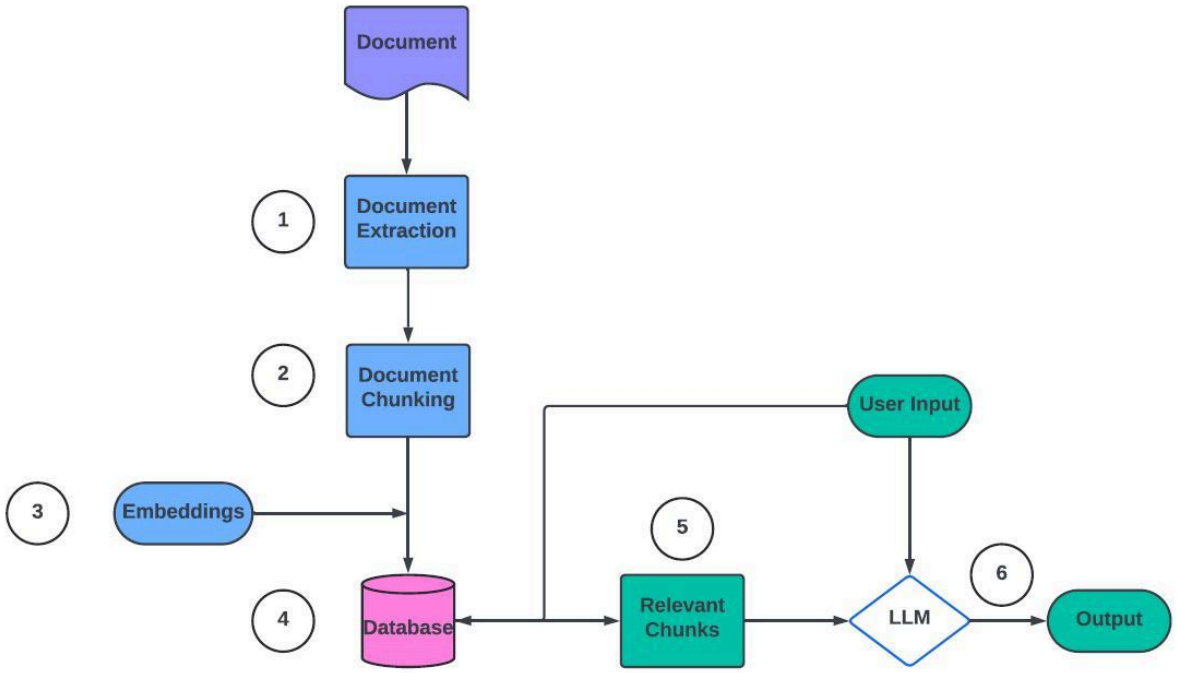
1. Presentation Layer (UI – Streamlit)
2. Processing Layer (Document ingestion & chunking)
3. Storage Layer (Vector DB – ChromaDB)
4. Retrieval Layer (Similarity search)
5. Generation Layer (LLM – Groq LLaMA)

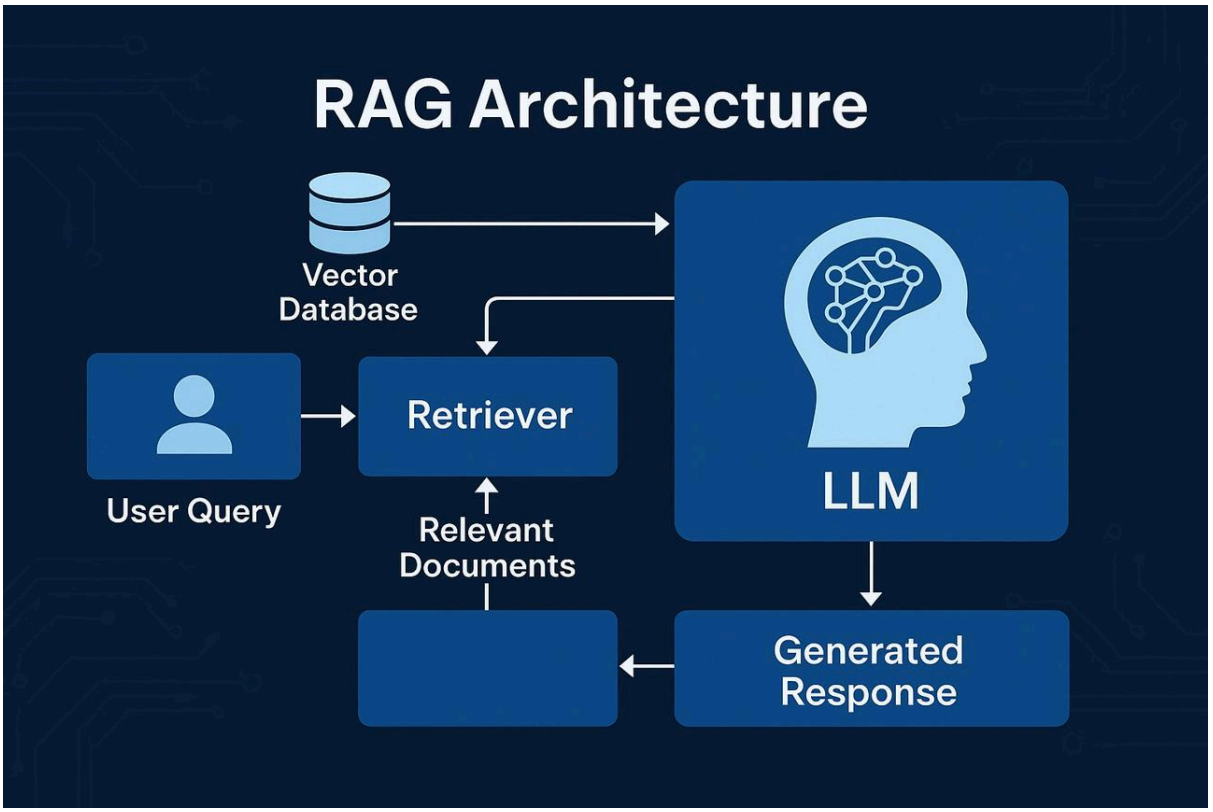
Each layer is loosely coupled and communicates via defined interfaces.

Architecture Diagram

RAG Architecture Model







2.1 Application Design

Frontend Layer

The frontend is developed using Streamlit and provides:

- File upload interface
- Interactive chat system
- Real-time response rendering

Backend Layer

The backend performs:

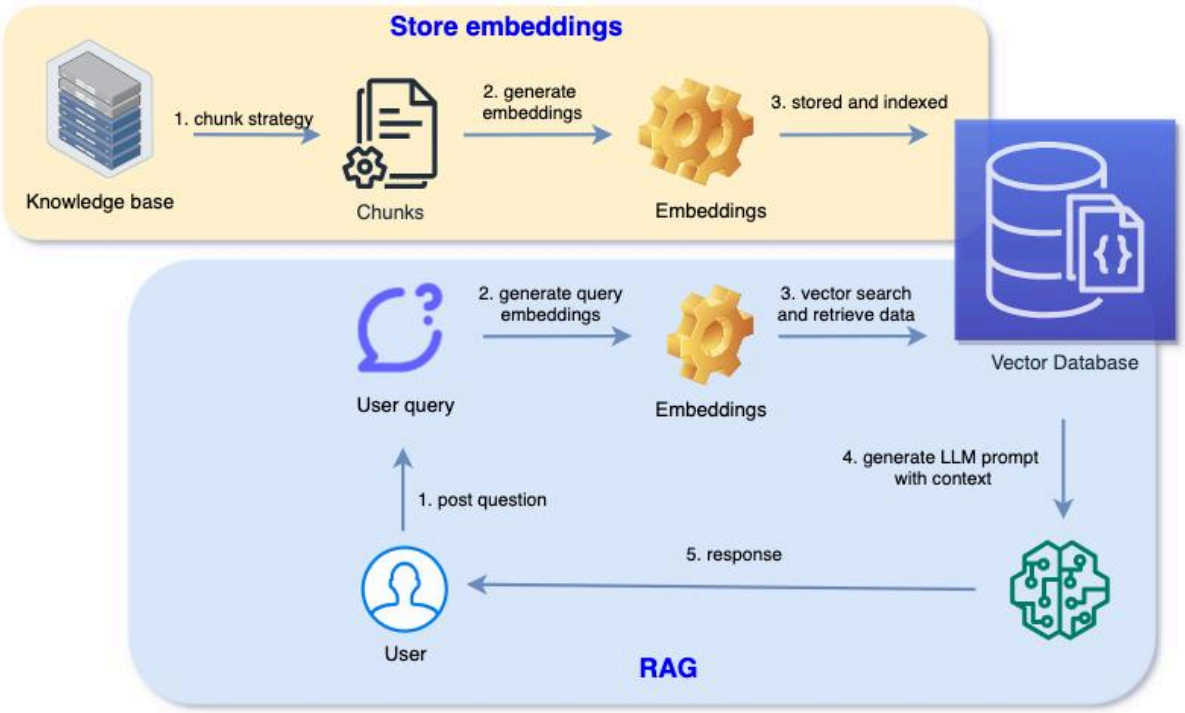
- Document ingestion and parsing
- Text preprocessing and chunking
- Embedding generation
- Vector storage and retrieval
- LLM interaction

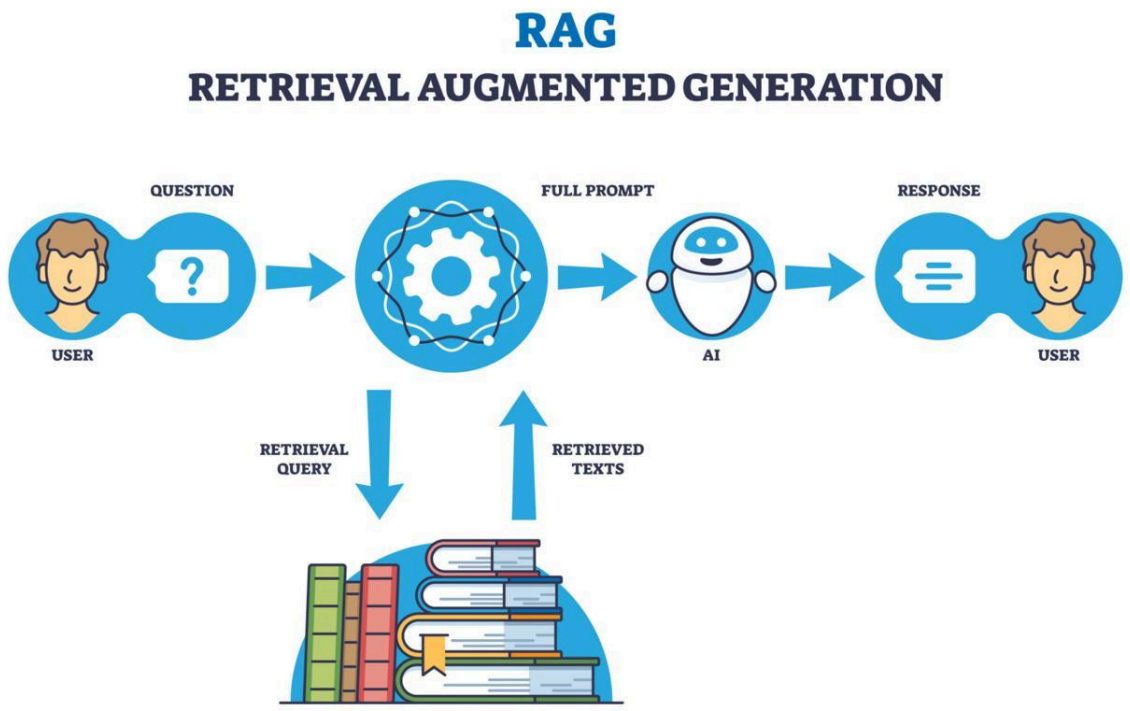
2.2 Process Flow

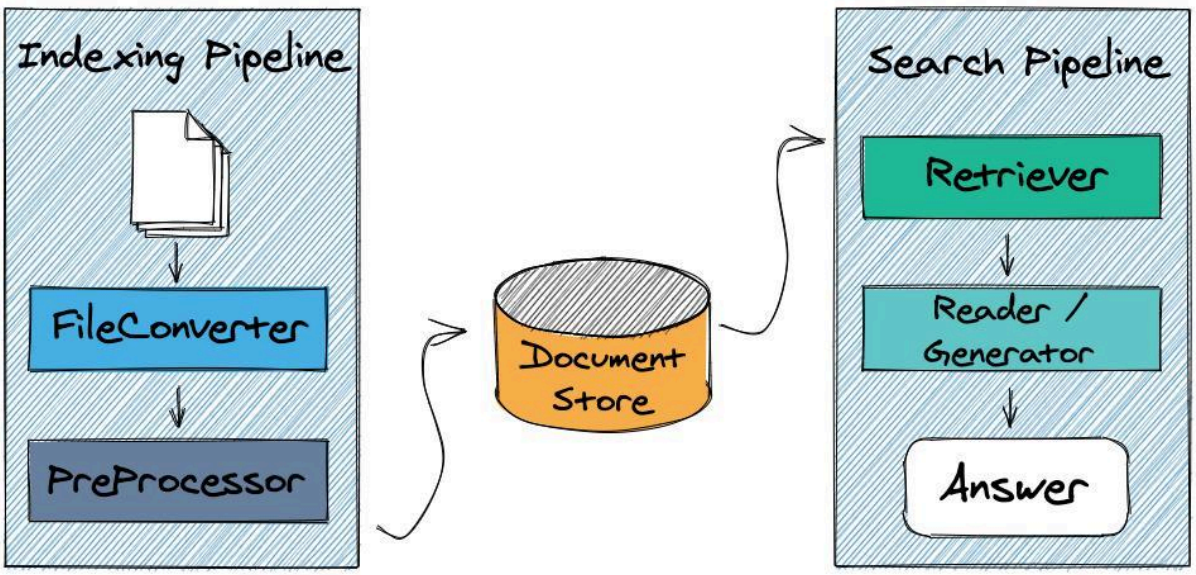
The system follows a structured pipeline:

1. User uploads logistics PDF documents
2. Documents are stored locally
3. Text is extracted using PDF loader
4. Text is split using RecursiveCharacterTextSplitter
5. Chunks are converted into embeddings using HuggingFace model
6. Embeddings are stored in ChromaDB
7. User submits a query
8. Query is converted into embedding
9. Similarity search retrieves top-k relevant chunks
10. Retrieved context is combined with query
11. Prompt is constructed
12. Prompt is sent to LLM (LLaMA via Groq API)
13. LLM generates grounded response
14. Response is displayed to user

Process Flow Diagram







2.3 Information Flow

The information flow describes how data moves:

- Input: User documents and queries
- Intermediate: Text chunks, embeddings, retrieved context
- Output: Context-aware response

Data transformation occurs at each stage:

- Raw text → Structured chunks
- Chunks → Vector embeddings
- Query → Embedded query
- Matching → Relevant context

2.4 Components Design

UI Component

Handles user interaction and display logic.

Document Processor

- Extracts text
- Performs chunking
- Maintains metadata

Embedding Engine

- Converts text into vector format
- Uses HuggingFace model

Vector Store (ChromaDB)

- Stores embeddings persistently
- Supports similarity search

Retriever

- Fetches top-k relevant chunks
- Based on vector similarity

LLM Engine

- Generates final answer
- Uses strict prompt for grounding

2.5 Key Design Considerations

Accuracy

Ensured through semantic retrieval and context-based generation

Performance

Optimized through efficient vector indexing

Scalability

Modular architecture supports extension

Cost Optimization

Local embeddings reduce API usage

Reliability

Strict prompt avoids hallucination

Maintainability

Independent modules for easy updates

2.6 API Catalogue

Function	Description
build_vector_store ()	Builds vector database
get_vector_store()	Loads existing DB
retrieve_context()	Retrieves relevant chunks

generate_answer( )	Generates final response

3. Data Design

3.1 Data Model

The system follows a transformation pipeline:

Document → Chunk → Embedding → Vector Storage

Each chunk includes:

- Text content
- Source metadata
- Page information

3.2 Data Access Mechanism

- Uses similarity search in vector DB
- Query embedding compared with stored embeddings
- Returns top-k matches

3.3 Data Retention Policies

- Documents stored locally
- Embeddings persist in ChromaDB
- Data can be deleted manually

3.4 Data Migration

- Data can be exported/imported
- Migration to cloud-based DB possible

4. Interfaces

- User Interface (Streamlit)
- LLM Interface (Groq API)
- Embedding Interface (HuggingFace model)

5. State and Session Management

- Managed using Streamlit session state
- Stores chat history temporarily
- No long-term user session storage

6. Caching

- Embeddings stored to avoid recomputation
- Improves response time
- Reduces processing overhead

7. Non-Functional Requirements

7.1 Security Aspects

- API keys stored securely in `.env`
- No external unauthorized access
- Local data handling ensures privacy

7.2 Performance Aspects

- Fast retrieval via vector DB
- Efficient chunking
- Low latency using Groq

8. References

- LangChain Documentation
- ChromaDB Documentation
- HuggingFace
- Groq API
- RAG Research Papers